

Content Indexing Modes — index side

(#70) Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: `index --mode full|search-only|metadata-only` recorded as the `content_mode` meta key, with per-mode storage (search-only = contentless FTS5, no stored plaintext; metadata-only = content never read) and `word_stats`-precedent read-time gates that reject or degrade clearly (issue #70, child of #39).

Architecture: One `ContentMode` enum flows `CLI` → `IndexOptions` → `SourceMeta` → `meta` table, mirroring `word_stats` exactly. Storage differences are decided at `create_v3` (FTS5 DDL variant) and in the per-entry pipeline (skip content read entirely for metadata-only; drop word-stat accumulation for search-only). Read paths load the mode once (absent key ⇒ `Full`, grandfathering every existing index) and gate: hard rejection for metadata-only content search, explicit degradation for search-only snippets/match-counts, mode-specific top messages. Master/dedup/JSON-fixture replication is #71, not here — but `search --json` gains its additive top-level `mode` field now because the single-index search arm is this child's code.

Tech Stack: Rust, rusqlite/FTS5 (contentless tables: `content=''` — tokens indexed, values discarded, `snippet()/highlight()` structurally unavailable), clap, existing golden-fixture harness untouched until #71.

Global Constraints

- **Additive-only JSON** (docs/CONTRACT.md): `search --json` may gain `mode`; existing keys keep names/types/meanings. Existing fixtures must stay green WITHOUT re-bless in this child (full-mode output byte-identical); new-mode fixtures land in #71.
- **Grandfathering** (docs/COMPATIBILITY.md): absent `content_mode` meta key ⇒ `Full`. No index or master rewrite to read anything.
- **Full mode is byte-identical to today** — every existing test passes unchanged; that is the regression gate for the whole child.
- **--no-word-stats stays orthogonal in full mode**; search-only forces word stats off (`word_freq` is the frequency-leakage surface); metadata-only trivially has none.
- **Safety invariant:** read-only tool; archives never touched.
- **Git discipline:** background autocheckpointer (60s) is the sole git writer; no manual add/commit in tasks.
- Gate per task: `cargo test`; end gate `cargo clippy --all-targets -- -D warnings + cargo fmt --check`.

Files

- Modify: `src/indexer.rs` — `ContentMode` enum, `IndexOptions.mode`, metadata-only fast path, search-only word-stat gate, summary echo
 - Modify: `src/store.rs` — `SourceMeta.mode`, `content_mode` meta write, FTS5 DDL variant, `content_mode()` reader
 - Modify: `src/searcher.rs` — mode loader + search/snippet/matches gates
 - Modify: `src/source_dir.rs` — same mode plumbing for directory sources
 - Modify: `src/cli.rs` — `--mode` on `IndexArgs`
 - Modify: `src/main.rs` — CLI wiring, read-time gates in Search/Top arms, summary print, `search --json mode` field
 - Create: `tests/modes.rs` — per-mode storage + gate tests, grandfather test
-

Task 1: `ContentMode` enum + CLI plumbing (no behavior change yet)

- [] **Step 1: failing test** (new `tests/modes.rs`):

```

//! Content indexing modes (#70, child of #39): per-mode storage and
//! read-time gates. Master/dedup replication is #71.

mod common;

use std::path::Path;

use backupsage::indexer::{self, ContentMode, IndexOptions};
use common::*;

fn index_with_mode(dir: &Path, name: &str, files: &[(&str, Vec<u8>)], mode:
ContentMode)
    -> (indexer::IndexSummary, rusqlite::Connection)
{
    let archive = write_archive(dir, name, &build_tar(files));
    let opts = IndexOptions { mode, ..IndexOptions::default() };
    let summary = indexer::run_index(&archive, None, &opts).unwrap();
    let conn = rusqlite::Connection::open(&summary.db_path).unwrap();
    (summary, conn)
}

#[test]
fn mode_is_recorded_in_meta_and_defaults_to_full() {
    let dir = tempfile::tempdir().unwrap();
    let files = [("a.txt", b"hello modeword".to_vec())];
    for (mode, expect) in [
        (ContentMode::Full, "full"),
        (ContentMode::SearchOnly, "search-only"),
        (ContentMode::MetadataOnly, "metadata-only"),
    ] {
        let (_s, conn) = index_with_mode(dir.path(), &format!("{expect}.tar"),
&files, mode);
        let got: String = conn
            .query_row("SELECT value FROM meta WHERE key='content_mode'", [], |r|
r.get(0))
            .unwrap();
        assert_eq!(got, expect);
    }
    assert_eq!(IndexOptions::default().mode, ContentMode::Full);
}

```

- [] **Step 2: implement.** `src/indexer.rs` (beside `IndexOptions`):

```

/// What an index stores about file CONTENT (#39). Recorded as the
/// `content_mode` meta key; an absent key on older indexes means `Full`.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
pub enum ContentMode {
    /// Everything: plaintext FTS, word stats, hashes, media metadata.
    #[default]
    Full,
    /// Contentless FTS (tokens indexed, text not stored – snippets and
    /// match counts are structurally unavailable), hashes and media
    /// metadata kept, word stats dropped. Tokens and their frequencies
    /// still leak; this is not encryption or a privacy boundary.
    SearchOnly,
    /// Content is never read: names, sizes, times, entry types only.
    /// No hashes, no FTS content, no media metadata. Content search is
    /// rejected; dedup cannot see these archives (#71 reports why).
    MetadataOnly,
}

impl ContentMode {
    pub fn as_str(self) -> &'static str {
        match self {
            ContentMode::Full => "full",
            ContentMode::SearchOnly => "search-only",
            ContentMode::MetadataOnly => "metadata-only",
        }
    }
    pub fn parse(s: &str) -> Option<Self> {
        match s {
            "full" => Some(ContentMode::Full),
            "search-only" => Some(ContentMode::SearchOnly),
            "metadata-only" => Some(ContentMode::MetadataOnly),
            _ => None,
        }
    }
}

```

IndexOptions gains pub mode: ContentMode (Default derives cover it). SourceMeta gains pub mode: ContentMode; create_v3 writes set_meta(&conn, "content_mode", meta.mode.as_str())?; beside word_stats. Both SourceMeta construction sites (indexer.rs create_db_with_fallback, source_dir.rs) pass opts.mode. src/cli.rs IndexArgs:

```

/// What to store about file content: full, search-only (contentless
/// FTS – searchable, no stored plaintext, no snippets), or
/// metadata-only (content never read; names/sizes/times only).
#[arg(long, default_value = "full",
    value_parser = ["full", "search-only", "metadata-only"])]
pub mode: String,

```

```
src/main.rs Index arm: mode: dedup-style parse –  
indexer::ContentMode::parse(&args.mode).expect("clap validated") into IndexOptions.
```

- [] **Step 3:** `cargo test` — new test green, all existing green (full still default everywhere).

Task 2: search-only storage — contentless FTS5, no word stats

- [] **Step 1: failing tests** (tests/modes.rs):

```
#[test]  
fn search_only_stores_no_plaintext_but_still_matches() {  
    let dir = tempfile::tempdir().unwrap();  
    let (_s, conn) = index_with_mode(  
        dir.path(),  
        "so.tar",  
        &[("doc.txt", b"secret plaintext with leakword inside".to_vec())],  
        ContentMode::SearchOnly,  
    );  
    // Tokens match...  
    let hits: i64 = conn  
        .query_row(  
            "SELECT COUNT(*) FROM files_fts WHERE files_fts MATCH 'leakword'",  
            [], |r| r.get(0),  
        )  
        .unwrap();  
    assert_eq!(hits, 1);  
    // ...but no shadow content table exists (contentless FTS5 stores none),  
    // so the plaintext is not recoverable from the index file.  
    let content_tables: i64 = conn  
        .query_row(  
            "SELECT COUNT(*) FROM sqlite_master WHERE name='files_fts_content'",  
            [], |r| r.get(0),  
        )  
        .unwrap();  
    assert_eq!(content_tables, 0, "contentless FTS must have no content shadow  
table");  
    // Hashes kept (dedup still works on search-only archives).  
    let (_, _, _, _, hash, _, _, _) = files_row(&conn, "doc.txt");  
    assert!(hash.is_some());  
    // Word stats dropped — the frequency surface.  
    let words: i64 = conn  
        .query_row("SELECT COUNT(*) FROM word_freq", [], |r| r.get(0))  
        .unwrap();  
    assert_eq!(words, 0);  
}
```

- [] **Step 2: implement.** `store::create_v3` FTS DDL becomes mode-dependent:

```

let fts_ddl = if meta.mode == ContentMode::SearchOnly {
    // Contentless: tokens are indexed at INSERT, values discarded.
    // snippet()/highlight() are structurally unavailable – read paths
    // gate on content_mode before using them.
    "CREATE VIRTUAL TABLE files_fts USING fts5(
        path, content, tokenize='unicode61', content='');"
} else {
    "CREATE VIRTUAL TABLE files_fts USING fts5(
        path, content, tokenize='unicode61');"
};

```

(keep the surrounding `execute_batch` structure; the insert path is unchanged — contentless tables accept the same INSERT and tokenize-then-discard). In `IndexRun::record`, gate word stats on `self.opts.word_stats && self.opts.mode != ContentMode::SearchOnly` — and force the recorded meta honest: in `create_v3`, write `word_stats` as `"0"` when mode is `SearchOnly` or `MetadataOnly` regardless of the flag.

- [] **Step 3:** cargo test green.

Task 3: metadata-only storage — content never read

- [] **Step 1: failing tests:**

```

#[test]
fn metadata_only_never_reads_content() {
    let dir = tempfile::tempdir().unwrap();
    let (summary, conn) = index_with_mode(
        dir.path(),
        "mo.tar",
        &[
            ("doc.txt", b"text that must never be read".to_vec()),
            ("pic.png", png_bytes(3, 64, 48)),
        ],
        ContentMode::MetadataOnly,
    );
    for path in ["doc.txt", "pic.png"] {
        let (etype, _kind, size, mtime, hash, phash, exif, _flg) = files_row(&conn,
path);
        assert_eq!(etype, "file");
        assert!(size > 0, "header metadata kept");
        assert!(mtime.is_some());
        assert!(hash.is_none(), "{path}: content must not be hashed");
        assert!(phash.is_none());
        assert!(exif.is_none());
    }
    let fts: i64 = conn
        .query_row(
            "SELECT COUNT(*) FROM files_fts WHERE files_fts MATCH 'read'",
            [], |r| r.get(0),
        )
        .unwrap();
    assert_eq!(fts, 0, "no content tokens in a metadata-only index");
    assert_eq!(summary.files_hashed, 0);
}

```

- [] **Step 2: implement.** In `index_tar`'s per-entry loop (and `source_dir.rs`'s equivalent), after the sparse/pax handling and before `process_reader: when opts.mode == ContentMode::MetadataOnly`, record a name-only row — same `EntryRecord` shape as the PAX-sparse branch (kind via `exif_date::media_kind(&entry_path)` for image/raw/video, else "binary", "empty" when size 0; `content_hash: None`, no phash/EXIF/dims, `fts_content: ""`, flags 0) and continue without reading the entry (the tar iterator skips unread data — proven by the #63 stream-sync probes). Counters: increment a per-run indexed count only; `files_hashed/files_indexed` stay 0.
- [] **Step 3:** cargo test green (existing suites untouched — the branch only fires for the new mode).

Task 4: read-time gates

- [] **Step 1: failing tests** (`tests/modes.rs` uses the CLI binary like `tests/cli.rs` — copy its `bin()/run_ok/stdout` helpers):

```
#[test]
fn metadata_only_rejects_search_clearly_and_search_only_degrades() {
    // metadata-only: `search` must exit 1 with a message naming the mode
    // and the fix, never print "No results".
    // search-only: `search` works; `--snippets` prints an explicit
    // degradation note and hits carry no match counts;
    // `search --json` carries "mode": "search-only" and "matches": null.
    // `top` on both modes explains itself (word_stats message precedent).
    // [build one archive per mode with a matching word, index via CLI
    // `--mode`, then assert on run() outputs – exact assertions:]
    // metadata-only search: exit 1, stderr contains "metadata-only" and "--mode"
    // search-only search: exit 0, stdout contains the hit path
    // search-only search --snippets: stdout contains "search-only" note line
    // search-only search --json: v["mode"]=="search-only", hits[0]
    ["matches"].is_null()
    // full search --json: v["mode"]=="full"
    // top on search-only: stdout contains "search-only"
}
```

• [] Step 2: implement.

- `src/store.rs: pub fn content_mode(conn) -> ContentMode` — `get_meta("content_mode")` parsed, `None/unknown` ⇒ `Full` (grandfather; unknown values from FUTURE versions also degrade to `Full` plus a `stderr` note, never a crash).
- `src/searcher.rs::search`: take the mode (loaded by callers once); when `SearchOnly`, skip the snippet SQL arm AND the highlight-based match count (hits get `matches: None` — change `SearchHit.matches` to `Option<usize>`; full mode wraps the existing count in `Some`, so full-mode JSON/text output is unchanged).
- `src/main.rs` Search arm (single index): after `open_index`, `match store::content_mode(&conn): MetadataOnly` ⇒ `bail!("this index is metadata-only – content search is unsupported; re-index with --mode full or --mode search-only")` (exit 1 via the normal error path); `SearchOnly + args.snippets` ⇒ `eprintln` note "search-only index: snippets are unavailable (no stored text)" and proceed without snippets. JSON object gains `"mode": mode.as_str()`; text hit table prints - where matches is `None` (adjust `print_search_table`).
- Top arm: before the word-stats check, `MetadataOnly/SearchOnly` ⇒ `println` mode-specific one-liner ("this index is metadata-only/search-only — word statistics are not stored; re-index with `--mode full`") and return `Ok(0)`.
- `search --all` (federated) is #71 EXCEPT it must not crash on a metadata-only child today: in `searcher::search_all`'s per-archive loop, load the mode; `MetadataOnly` ⇒ `push (label, "metadata-only – content search unsupported")` onto `skipped` and `continue` (exit-2 semantics fall out of the existing `has skips` return). This is the minimal correctness piece; fixtures/JSON mode fields wait for #71.
- [] Step 3: cargo test green.

Task 5: grandfather + summary echo

• [] Step 1: failing test:

```
#[test]
fn old_index_without_mode_key_reads_as_full() {
    let dir = tempfile::tempdir().unwrap();
    let (_s, conn) = index_with_mode(
        dir.path(), "old.tar",
        &[("a.txt", b"grandfather word".to_vec())], ContentMode::Full,
    );
    let db = dir.path().join("old.tar.db");
    drop(conn);
    // Simulate a pre-#70 index: the key simply doesn't exist.
    let conn = rusqlite::Connection::open(&db).unwrap();
    conn.execute("DELETE FROM meta WHERE key='content_mode'", []).unwrap();
    assert_eq!(backupsage::store::content_mode(&conn), ContentMode::Full);
    // And search still works end-to-end through the CLI.
    // [run_ok(&["search", "grandfather", "-i", db_str]) → exit 0, hit shown]
}
```

- [] **Step 2:** summary echo — `IndexSummary` gains `pub mode: String` (set from opts at run start); `print_index_summary` prints `Mode : search-only` (contentless FTS – no stored plaintext) / `metadata-only` (content not read) lines only when not full.
- [] **Step 3:** cargo test green.

Task 6: gate + review + ship

- [] Full gate: `cargo test && cargo clippy --all-targets -- -D warnings && cargo fmt --check`.
- [] Adversarial verify (session-model agent, read-only): attack the contentless-FTS claim (is plaintext truly unrecoverable from the `.db` — check ALL fts5 shadow tables, not just `_content`), the metadata-only never-reads claim (any path where `process_reader` still runs), full-mode byte-identity (diff a full-mode index built pre/post change), gate bypasses (`search --all`, `discover_db_path` fallbacks, snippets via JSON).
- [] Fix findings, stop checkpointer, final commit as `claude_2010`, push, PR "v1.0.2: content indexing modes — index side (#70)", CI, merge (keep branch), close #70.

Self-Review

- Spec coverage: #70 ACs — CLI/schema/default ✓ (T1, T5); no recoverable plaintext + still matches ✓ (T2); never reads + clear rejection ✓ (T3, T4); full byte-identical ✓ (every task's regression gate + T6 verify). #39 ACs deferred to #71: master/dedup/JSON fixtures/migration fixture/README+ADR.
- Placeholder scan: T4/T5 Step-1 tests carry bracketed CLI-assertion sketches pointing at tests/cli.rs's exact helper trio to copy — deliberate, the helpers are 15 lines in that file.
- Type consistency: `ContentMode` names identical across tasks; `SearchHit.matches: Option<usize>` change named in T4 where it happens.

Signed: thomas2025 · 2026-08-02T05:35:00-04:00